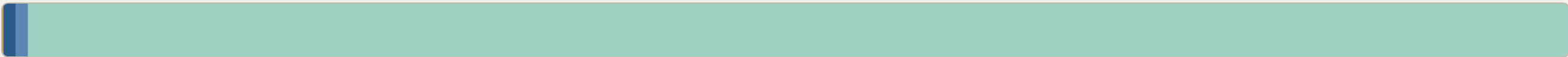


FAT32, byte by byte

Where every byte of a 34 MiB FAT32 volume lives, and which ones change when two text files are written, one of them grows, and the disk is defragmented. Every value on these boards is read from the images the demo script builds.

THE WHOLE DEVICE, TO SCALE · 69,632 SECTORS OF 512 BYTES



The reserved area and both FATs are the slivers at the left edge: 1,104 sectors, 1.6% of the disk. Everything else is data clusters. 0x0 → 0x2200000

THE SAME DEVICE, ZOOMED INTO ITS STRUCTURES · NOT TO SCALE



Finding anything

Two formulas locate every structure on these boards.

```
FAT entry N → 0x4000 + 4·N
copy in FAT 2 → 0x47000 + 4·N
cluster N → 0x8A000 + (N - 2)·0x200
```

Clusters are numbered from 2 because FAT entries 0 and 1 are used for bookkeeping. So cluster 3 starts at 0x8A200 and its FAT entry is at 0x400C.

Little-endian numbers

Multi-byte values are stored least significant byte first. Read the bytes right to left.

on disk	00	10	01	00
reversed	00	01	10	00
value	0x00011000 = 69,632			

This is the total sector count at boot sector offset 0x20.

Reading HELLO.TXT

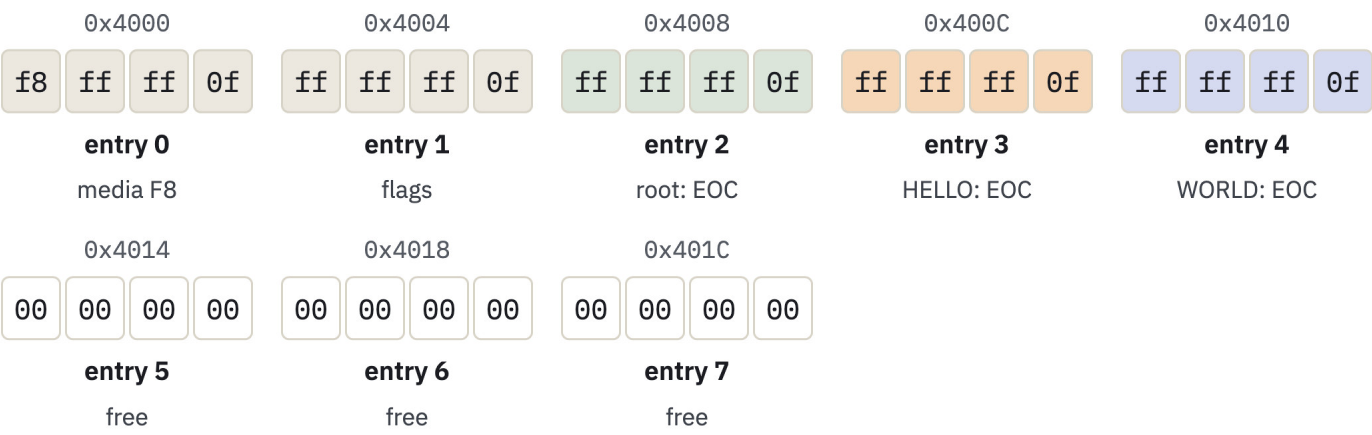
- 1 Read the boot sector**
LBA 0 gives cluster size, FAT location and root cluster (2).
- 2 Read the root directory**
Cluster 2 at 0x8A000. Scan 32-byte entries for HELLO TXT.
- 3 Take the start and size**
The entry says: first cluster 3, size 14 bytes.
- 4 Read cluster 3**
At 0x8A000 + (3 - 2) × 0x200 = 0x8A200.
- 5 Look up FAT[3]**
At 0x4000 + 4 × 3 = 0x400C: ff ff ff 0f, end of chain. Done.

FAT entries and directory entries

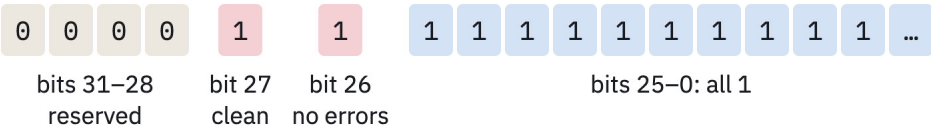
There are no inodes. A file is a 32-byte directory entry, which records its name, size, timestamps and first cluster, plus a chain of 4-byte FAT entries linking the rest of its clusters.

The FAT: one 4-byte link per cluster

FAT 1 at 0x4000, stage 1. The entry for cluster N holds the number of the next cluster in the same file. Entries 0 and 1 are reserved, which is why the first data cluster is number 2.



ENTRY 1 = 0X0FFFFFFF, BIT BY BIT



A driver clears bit 27 while the volume is mounted and sets it on a clean dismount. Bit 26 is cleared if a disk I/O error occurs.

What an entry's value means

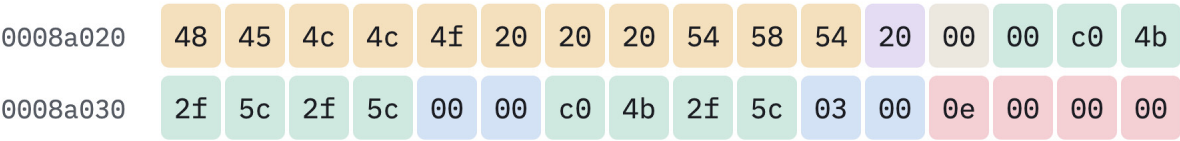
Only the low 28 bits are the value. The top 4 bits are reserved and must be kept when an entry is rewritten.

0x00000000	Free cluster
0x00000002 – 0x0FFFFFFF	Next cluster in this file
0x0FFFFFF7	Bad cluster, never allocate
0x0FFFFFF8 – 0x0FFFFFFF	End of chain: last cluster of the file

Entry 0 holds 0x0FFFFFF8, whose low byte repeats the media descriptor F8. FAT 2 at 0x47000 holds identical bytes.

A directory entry: HELLO.TXT

32 bytes in the root directory, cluster 2, at 0x8A020.



- Name

Attributes

Times and dates

First cluster

File size

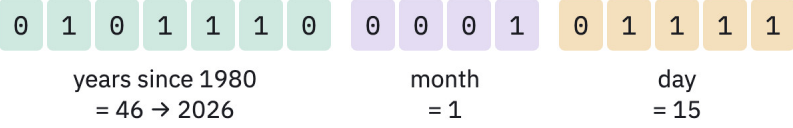
Reserved, always zero

Packed times and dates

c0 4b → time 0x4BC0 → 09:30:00



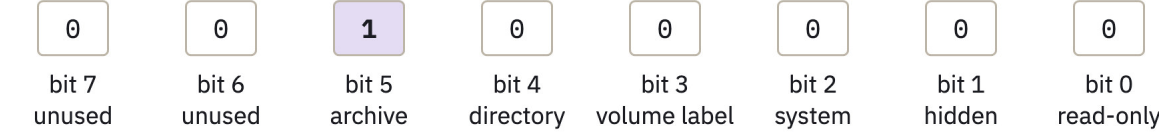
2f 5c → date 0x5C2F → 2026-01-15



Seconds are stored halved, so FAT timestamps have two-second resolution. Years run from 1980 to 2107.

OFFSET	BYTES	FIELD	VALUE	NOTE
0x00	48 45 4c 4c 4f 20 20 20	Name	"HELLO"	Space-padded; no dot stored
0x08	54 58 54	Extension	"TXT"	
0x0B	20	Attributes	0x20	Archive bit set
0x0C	00	Reserved (case flags)	0	Used by Windows NT for lowercase names
0x0D	00	Created, 1/100 s	0	
0x0E	c0 4b	Created time	09:30:00	0x4BC0
0x10	2f 5c	Created date	2026-01-15	0x5C2F
0x12	2f 5c	Accessed date	2026-01-15	Date only, no time
0x14	00 00	First cluster, high 16	0	Added for FAT32
0x16	c0 4b	Modified time	09:30:00	
0x18	2f 5c	Modified date	2026-01-15	
0x1A	03 00	First cluster, low 16	3	Combined: (hi << 16) lo
0x1C	0e 00 00 00	File size	14 bytes	Rest of the cluster is slack

Attribute byte 0x20



Long file names use attribute 0x0F (read-only + hidden + system + volume label), a combination no real file can have, so older drivers skip them.

The first name byte is also a flag

0x00	End of directory: stop reading
0xE5	Deleted entry: slot is free
0x05	Name really starts with byte 0xE5
0x2E	"." or "." entry in a subdirectory

Write, grow, defragment: the bytes that change

The same metadata in each image. Highlighted bytes differ from the previous stage. FAT 2 and the backup FSInfo sector always receive identical changes.

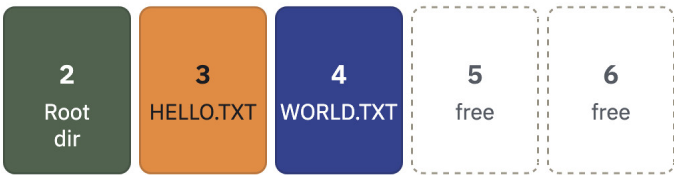
05 changed since the previous stage

STAGE 1 · fat32_1_two_files.img

Two files written

Each file fits in one 512-byte cluster, so each chain is a single end-of-chain entry.

CLUSTERS 2–6 · 0X8A000



HELLO.TXT 3

WORLD.TXT 4

FAT 1 ENTRIES

FAT[2]	0x4008	ff	ff	ff	0f	end of chain
FAT[3]	0x400C	ff	ff	ff	0f	end of chain
FAT[4]	0x4010	ff	ff	ff	0f	end of chain
FAT[5]	0x4014	00	00	00	00	free
FAT[6]	0x4018	00	00	00	00	free

ROOT DIRECTORY

HELLO.TXT entry

modified	0x8A036	c0	4b	09:30:00
cluster	0x8A03A	03	00	first cluster 3
size	0x8A03C	0e	00	14 bytes

WORLD.TXT entry

modified	0x8A056	c0	4b	09:30:00
cluster	0x8A05A	04	00	first cluster 4
size	0x8A05C	19	00	25 bytes

FSINFO

free	0x3E8	ad	0b	01	00	68,525 clusters
hint	0x3EC	04	00	00	00	cluster 4

The next-free hint says 4: the most recently allocated cluster. The driver starts its next search just after it.

STAGE 2 · fat32_2_grown.img

HELLO.TXT grows

Now 694 bytes, it needs a second cluster. Cluster 4 is taken, so it gets 5: the file is fragmented.

CLUSTERS 2–6 · 0X8A000



HELLO.TXT 3 → 5

WORLD.TXT 4

FAT 1 ENTRIES

FAT[2]	0x4008	ff	ff	ff	0f	end of chain
FAT[3]	0x400C	05	00	00	00	next → 5
FAT[4]	0x4010	ff	ff	ff	0f	end of chain
FAT[5]	0x4014	ff	ff	ff	0f	end of chain
FAT[6]	0x4018	00	00	00	00	free

ROOT DIRECTORY

HELLO.TXT entry

modified	0x8A036	a5	4d	09:45:10
cluster	0x8A03A	03	00	first cluster 3
size	0x8A03C	b6	02	694 bytes

WORLD.TXT entry

modified	0x8A056	c0	4b	09:30:00
cluster	0x8A05A	04	00	first cluster 4
size	0x8A05C	19	00	25 bytes

FSINFO

free	0x3E8	ac	0b	01	00	68,524 clusters
hint	0x3EC	05	00	00	00	cluster 5

FAT[3] now links to 5 and FAT[5] ends the chain. The first cluster in the entry never changes when a file grows.

STAGE 3 · fat32_3_defragmented.img

Defragmented

Three cluster moves make both files contiguous. See the next board for each step.

CLUSTERS 2–6 · 0X8A000



HELLO.TXT 3 → 4

WORLD.TXT 5

FAT 1 ENTRIES

FAT[2]	0x4008	ff	ff	ff	0f	end of chain
FAT[3]	0x400C	04	00	00	00	next → 4
FAT[4]	0x4010	ff	ff	ff	0f	end of chain
FAT[5]	0x4014	ff	ff	ff	0f	end of chain
FAT[6]	0x4018	00	00	00	00	free

ROOT DIRECTORY

HELLO.TXT entry

modified	0x8A036	a5	4d	09:45:10
cluster	0x8A03A	03	00	first cluster 3
size	0x8A03C	b6	02	694 bytes

WORLD.TXT entry

modified	0x8A056	c0	4b	09:30:00
cluster	0x8A05A	05	00	first cluster 5
size	0x8A05C	19	00	25 bytes

FSINFO

free	0x3E8	ac	0b	01	00	68,524 clusters
hint	0x3EC	05	00	00	00	cluster 5

FAT[4] and FAT[5] hold the same bytes as in stage 2, but now end different files. Cluster 6 is free but still holds a copy of WORLD.TXT.

Defragmenting: three moves, four steps each

Two clusters can't be swapped in place, so the defragmenter routes WORLD.TXT through free cluster 6. Every move repeats the same four steps, in an order that survives a power failure at any point. FAT 2 at 0x47000 + 4·N receives every FAT write too.

1 • Copy

Copy the data to a free cluster.

IF POWER FAILS AFTER THIS STEP

The FAT still calls the destination free.
Nothing has changed.

2 • Claim

Copy the source's FAT entry into the destination's.

IF POWER FAILS AFTER THIS STEP

The destination is allocated but unreferenced: a lost cluster, reclaimed by fsck.

3 • Repoint

Point the directory entry or previous FAT entry at the destination.

IF POWER FAILS AFTER THIS STEP

The new copy is live. The old one is now the lost cluster.

4 • Free

Set the source FAT entry to 0.

IF POWER FAILS AFTER THIS STEP

Consistent. The old data stays in the cluster until overwritten.

Move 1 • WORLD.TXT out of the way

Cluster 4 is needed for HELLO.TXT, so WORLD.TXT goes to free cluster 6 first.

BEFORE



AFTER



1	Copy data	cluster 4 → cluster 6	512 bytes	0x8A400 → 0x8A800
2	Set FAT[6]	end of chain	ff ff ff 0f	0x4018
3	Repoint	WORLD.TXT first cluster 4 → 6	06 00	0x8A05A
4	Free FAT[4]	mark free	00 00 00 00	0x4010

Move 2 • HELLO.TXT cluster #2 into place

Its second cluster moves from 5 to the now-free cluster 4.

BEFORE



AFTER



1	Copy data	cluster 5 → cluster 4	512 bytes	0x8A600 → 0x8A400
2	Set FAT[4]	end of chain	ff ff ff 0f	0x4010
3	Repoint	FAT[3]: next is 4, was 5	04 00 00 00	0x400C
4	Free FAT[5]	mark free	00 00 00 00	0x4014

Move 3 • WORLD.TXT into place

WORLD.TXT moves from its temporary cluster 6 to cluster 5.

BEFORE



AFTER



1	Copy data	cluster 6 → cluster 5	512 bytes	0x8A800 → 0x8A600
2	Set FAT[5]	end of chain	ff ff ff 0f	0x4014
3	Repoint	WORLD.TXT first cluster 6 → 5	05 00	0x8A05A
4	Free FAT[6]	mark free	00 00 00 00	0x4018

The net change, stage 2 → 3

FAT[3]	0x400C	04 00 00 00	next → 4 (was 5)
FAT[4]	0x4010	ff ff ff 0f	end of chain, now HELLO.TXT
FAT[5]	0x4014	ff ff ff 0f	end of chain, now WORLD.TXT
WORLD	0x8A05A	05 00	first cluster 5 (was 4)

Only FAT[3] and WORLD.TXT's first cluster differ. FAT[4] and FAT[5] end up byte-identical but belong to different files: the FAT never records which file owns a chain.

What freeing leaves behind

Cluster 6 is free again, since its FAT entry is 00 00 00 00, but its 512 bytes still hold the temporary copy of WORLD.TXT:

0008a800 54 68 69 73 20 69 73 20 74 68 65 20 73 65 63 6f

Freeing only rewrites a FAT entry. Deleted files work the same way, which is why undelete tools work and why deleting doesn't erase data. HELLO.TXT's directory entry never changes: moving clusters doesn't change a file's start, size or timestamps.